

Analisi distribuita di co-occorrenza di terremoti

Corso: Scalable and Cloud Programming – A.A. 2025/2026

Studente: Rosario Zaccone

Repository: github.com/rosario-zaccone/quake-cooccurrence

Obiettivo

Il progetto richiede di individuare, su un dataset di terremoti, la coppia di località distinte che presenta il massimo numero di co-occorrenze giornaliere. Ogni evento viene normalizzato approssimando latitudine e longitudine alla prima cifra decimale e usando solo la data del timestamp. L'output finale è composto dalla coppia di coordinate più frequente e dalla lista ordinata delle date in cui le due località co-occorrono.

Come stack tecnologico è stato usato Apache Spark insieme a Google Cloud Dataproc con macchine `n2-standard-4`, variando sia il numero di worker sia il numero di partizioni usato dal job.

Implementazione proposta

L'algoritmo generale è il seguente:

1. lettura del dataset da `gs://<bucket>/dataset.csv`;
2. normalizzazione di data e coordinate;
3. raggruppamento per data e rimozione dei duplicati;
4. generazione di tutte le coppie distinte di località presenti nello stesso giorno;
5. aggregazione per coppia di località;
6. selezione della coppia più frequente;
7. ordinamento crescente delle date.

Sono state implementate cinque varianti, che variano nell'implementare l'algoritmo principale (da passo 3 fino alla fine, poiché il cleaning dei dati è identico in ogni versione), per confrontare diverse metodologie e capire quale soluzione scali meglio.

Preparazione dei dati

Il dataset è fornito sottoforma di CSV, risiede su Google Cloud Storage. Come primo passo viene convertito in un RDD di coppie:

```
(date, Coordinate(latRounded, lonRounded))
```

La data viene estratta dai primi 10 caratteri del campo `date`, mentre le coordinate vengono arrotondate alla prima cifra decimale. Il dataset viene ripartizionato con:

```
repartition(sc.defaultParallelism * partitionMultiplier)
```

Il parametro `partitionMultiplier` consente di confrontare configurazioni con partizionamento pari a 2x, 3x e 4x rispetto al parallelismo di default del cluster.

Solver1

Il primo approccio costituisce la base di partenza, sulla quale sono state fondate le altre versioni di implementazione dell'algoritmo che mirano a migliorarlo. Il raggruppamento per data (passo 3) è realizzato tramite `groupByKey()`, seguito da una conversione in `Set` per rimuovere in modo elegante i duplicati.

Le coppie di località (passo 4) sono enumerate con la rispettiva data tramite una `for-comprehension` con doppio indice, mentre l'aggregazione per coppia (passo 5) usa `groupByKey`: entrambe le operazioni non eseguono alcuna aggregazione parziale, rendendo questa variante potenzialmente la più onerosa. La selezione della coppia più frequente (passo 6) è affidata a `reduce`, che propaga il confronto tra coppie adiacenti attraverso le partizioni fino a restituire il risultato globale.

Solver2

Il secondo approccio introduce la prima ottimizzazione significativa: il raggruppamento per data (passo 3) viene sostituito da `aggregateByKey()`, che esegue la deduplicazione delle coordinate già a livello di partizione locale, riducendo sensibilmente il volume di dati trasferiti durante lo shuffle. Il resto della pipeline rimane invariato rispetto a Solver1: la generazione delle coppie usa la stessa for-comprehension con indici, e l'aggregazione per coppia è ancora basata su `groupByKey`.

Solver3

Il terzo approccio mantiene `aggregateByKey` per il passo 3 e introduce una variante per la generazione delle coppie (passo 4): al posto della for-comprehension con doppio indice, viene utilizzato il metodo `.combinations(2)` della libreria standard di Scala. Questa soluzione è più leggibile e dichiarativa, ma genera oggetti intermedi aggiuntivi (`Seq` per ogni coppia), il che può comportare un overhead di allocazione rispetto all'approccio con indici espliciti su dataset di grandi dimensioni. L'aggregazione per coppia rimane basata su `groupByKey`, analogamente a Solver1 e Solver2.

Solver4

Il quarto approccio cerca di ottimizzare sia la generazione delle coppie (passo 4) sia l'aggregazione per coppia (passo 5). La generazione è riscritta con un doppio ciclo `while` esplicito che popola un `ArrayBuffer` mutabile: si evita così l'allocazione di oggetti intermedi tipica delle for-comprehension e di `.combinations`. L'aggregazione per coppia usa `aggregateByKey()`, che deduplica le date già durante la fase di combine locale.

Solver5

La generazione delle coppie (passo 4) riprende il doppio ciclo `while` con `ArrayBuffer` di Solver4, ma viene eseguita all'interno di `mapPartitions`: anziché invocare una funzione per ogni elemento dell'RDD, l'elaborazione avviene per intera partizione. L'aggregazione per coppia usa `aggregateByKey()` come in Solver4. La riduzione finale (passo 6) sostituisce `reduce` con `treeReduce`, che organizza la riduzione in un albero bilanciato distribuito tra gli esecutori anziché convogliare tutti i risultati parziali verso un unico nodo: questo evita il collo di bottiglia sul driver in presenza di un numero elevato di partizioni e rende la fase di selezione del massimo più scalabile.

Ambiente sperimentale

Le prove sono state eseguite su Dataproc con la seguente configurazione:

Parametro	Valore
Regione	europa-west1
Tipo macchina	n2-standard-4
Worker testati	2, 3, 4
Partizionamento	2x, 3x, 4x rispetto a <code>sc.defaultParallelism</code>
Ripetizioni sperimentali	diverse run per combinazione solver/worker/partizioni

Gli script in `scripts/` automatizzano creazione dei bucket, upload del dataset, creazione del cluster, sottomissione dei job, raccolta dei log e generazione dei grafici. Le metriche aggregate sono salvate in `results/all_results.csv`, le metriche di una singola esecuzione sono in `results/results.csv`.

Risultati

La tabella seguente riporta il tempo medio in minuti e secondi calcolato su diverse run per ciascuna configurazione.

Solver	Worker	p2x	p3x	p4x
Solver1	2	9 min 36 sec	9 min 13 sec	5 min 40 sec
Solver1	3	10 min 40 sec	6 min 44 sec	5 min 18 sec
Solver1	4	9 min 48 sec	6 min 54 sec	4 min 19 sec
Solver2	2	9 min 53 sec	9 min 15 sec	5 min 50 sec

Solver	Worker	p2x	p3x	p4x
Solver2	3	10 min 49 sec	6 min 45 sec	5 min 7 sec
Solver2	4	10 min 32 sec	6 min 58 sec	3 min 56 sec
Solver3	2	11 min 38 sec	10 min 52 sec	7 min 3 sec
Solver3	3	12 min 12 sec	7 min 58 sec	6 min 35 sec
Solver3	4	11 min 57 sec	7 min 57 sec	5 min 23 sec
Solver4	2	27 min 27 sec	21 min 53 sec	17 min 42 sec
Solver4	3	25 min 10 sec	17 min 10 sec	13 min 48 sec
Solver4	4	22 min 3 sec	15 min 51 sec	10 min 27 sec
Solver5	2	24 min 52 sec	25 min 27 sec	17 min 30 sec
Solver5	3	24 min 32 sec	17 min 16 sec	15 min 15 sec
Solver5	4	20 min 56 sec	15 min 18 sec	12 min 12 sec

La coppia di coordinate con il maggior numero di co-occorrenze è composta dai punti (longitude = 38.8, latitude = -122.8) e (longitude = 38.8, latitude = -122.7), con ben 10,032 date associate.

Analisi prestazioni

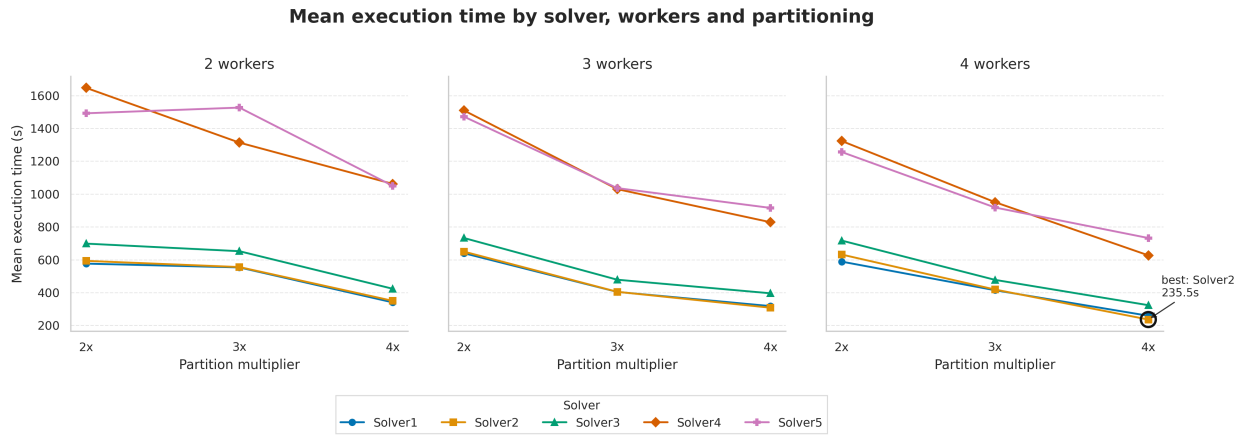


Figure 1: Grafico prestazioni

Dalla Figura 1 si osservano dei tempi di esecuzioni molto diversi. Si osserva che *Solver1* e *Solver2*, su questo dataset, risultano più performanti rispetto agli altri algoritmi.

Solver4 e *Solver5* presentano i tempi di esecuzione peggiori. Si distinguono dagli altri algoritmi per l'uso di *aggregateByKey* nella fase di raggruppamento delle coppie: invece di raccogliere i valori grezzi tramite *groupByKey*, costruiscono il Set di date già localmente su ogni nodo, riducendo il volume dei dati trasferiti attraverso la rete. Sul dataset attuale questo overhead non ripaga, poiché supera il risparmio ottenuto dallo shuffle ridotto. Al crescere del dataset però, si ha un numero di coppie maggiore e in questo caso il *groupByKey* dei primi solver diventerebbe un collo di bottiglia, mentre l'approccio di *Solver4* e *Solver5* si rivelerebbe vincente, come suggerito anche dal loro speedup superiore.

Osservando invece come varia il tempo di esecuzione in base al *partition multiplier*, si nota che passando da 2x a 3x e soprattutto a 4x, i tempi di esecuzione tendono a diminuire, effetto che si nota soprattutto nei solver più lenti. Questo perché aumentando il numero di partizioni andiamo a suddividere il dataset in chunk più piccoli, distribuendo meglio il carico di lavoro. Per *Solver4* e *Solver5*, che eseguono computazioni costose localmente, avere partizioni di piccolo volume significa elaborare meno coppie per task, e un parallelismo più efficace.

L'aumento del numero di worker non produce sempre un miglioramento uniforme, si nota soprattutto nel caso con *partition multiplier* 2x. Possiamo dedurre che con una granularità di partizionamento molto bassa il carico non viene distribuito in modo efficace, oppure il lavoro di coordinamento dei worker diventa rilevante rispetto al lavoro utile svolto. Nei casi di *partition multiplier* 3x e 4x possiamo vedere come l'aumento dei worker risulti in un tempo di esecuzione migliore, probabilmente dato da un buon bilanciamento fra worker disponibili e partizioni. La configurazione migliore si osserva per *Solver2* con *partition multiplier* 4x e 4 worker, che raggiunge il tempo medio più basso. Questo conferma che i solver più efficienti beneficiano maggiormente di una combinazione tra maggiore parallelismo e maggiore granularità del partizionamento.

Analisi scalabilità

Lo speedup è stato calcolato usando come baseline la configurazione con 2 worker ed è definito come:

$$S(w) = \frac{T_2}{T_w}$$

dove T_2 è il tempo medio di esecuzione ottenuto con 2 worker, mentre T_w è il tempo medio di esecuzione ottenuto con w worker.

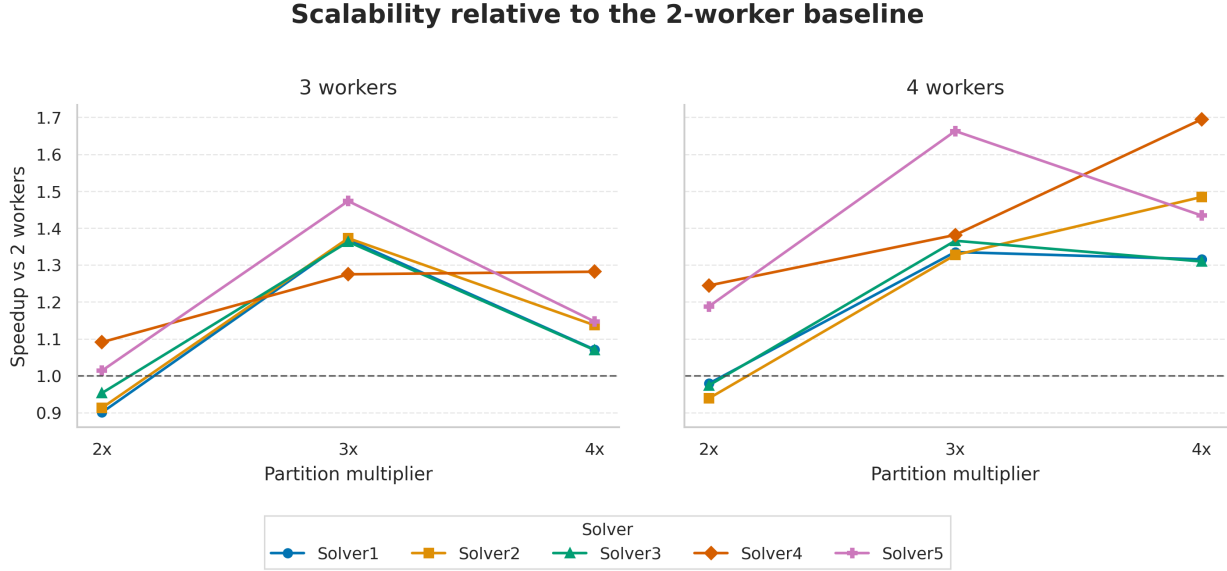


Figure 2: Grafico speedup

Dalla Figura 2 si può notare l'andamento dello speedup al variare dei solver, del *partition multiplier* e dei worker. Con 3 worker si nota subito che lo speedup migliora con un *partition multiplier* di 3x, mentre con 2x molti solver stanno sotto la baseline, segno che in quei casi non c'è un buon sfruttamento del parallelismo. Si nota che con 4x lo speedup tende a diminuire per quasi tutti i solver.

Con 4 worker si hanno risultati più soddisfacenti, quasi tutti i solver superano la baseline già con 2x, e gli speedup aumentano notevolmente con 3x e 4x. In particolare si nota un aumento di speedup in *Solver4* e *Solver2* correlato all'aumento del *partition multiplier*.

I risultati indicano che la configurazione più efficace dipende sia dal numero di worker sia dal numero di partizioni. Aumentare i worker migliora le prestazioni, ma il beneficio dipende dalla capacità dell'algoritmo di sfruttare il parallelismo. Il moltiplicatore 3x appare una scelta equilibrata, mentre 4x può essere vantaggioso solo per alcuni solver, come *Solver4*, ma penalizzante per altri. Questo conferma che la scelta del numero di partizioni è un parametro rilevante per la scalabilità su Spark. *Solver4* e *Solver5* appaiono gli algoritmi più scalabili.

La strong scaling efficiency misura quanto contribuisce in media ogni worker all'aumento delle prestazioni, mantenendo fissa la dimensione del problema. È definita come:

$$E(w) = \frac{S(w)}{w}$$

dove $S(w)$ è lo speedup ottenuto con w worker. Il valore di $E(w)$ varia tra 0 e 1, dove 1 rappresenta il caso ideale.

Le configurazioni con *partition multiplier* 2x presentano generalmente efficienze non molto entusiasmanti, specialmente con 4 worker, come mostrato in Figura 3. Si può supporre che in questi casi l'aggiunta di un nodo non venga sfruttata pienamente e non venga compensato l'overhead di parallelizzazione.

Le efficienze migliori si osservano con *partition multiplier* 3x e 3 worker. In questa configurazione diversi solver mostrano valori molto soddisfacenti. Si nota in particolare *Solver5* che raggiunge un'efficienza pari a

Strong scaling efficiency

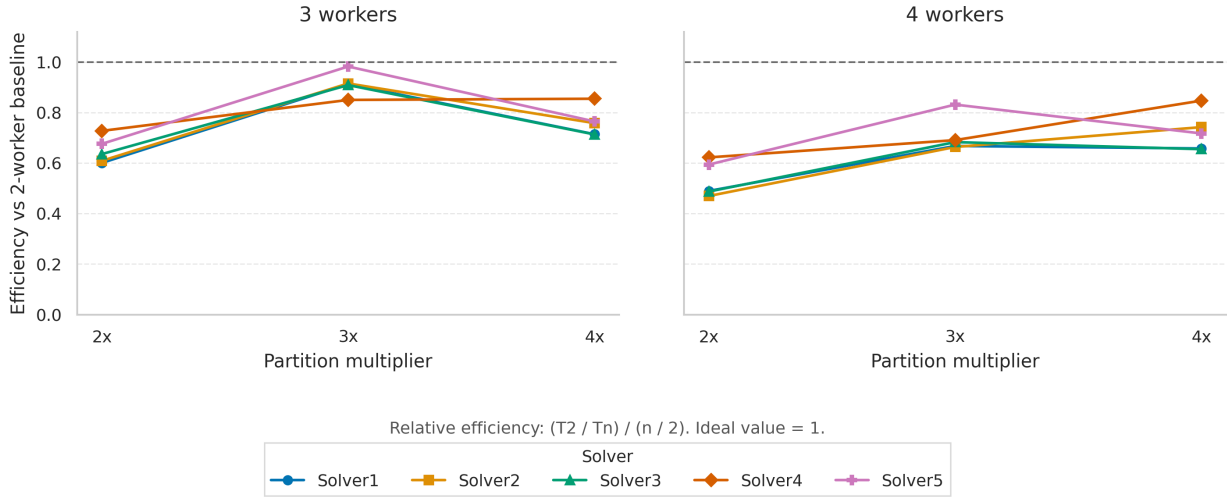


Figure 3: Grafico strong scaling efficiency

circa 0.98, indicando che il terzo worker contribuisce quasi idealmente al miglioramento delle prestazioni rispetto alla baseline. Anche *Solver1*, *Solver2* e *Solver3* mostrano efficienze superiori a 0.9 con 3x e 3 worker.

Con 4 worker, l'efficienza tende invece a diminuire rispetto alla configurazione con 3 worker. Si può supporre che l'aumento del parallelismo introduce un overhead maggiore e che non tutto il lavoro può essere distribuito in modo perfettamente bilanciato. Tuttavia, con *partition multiplier* 4x, alcuni solver mantengono valori di efficienza buoni anche con 4 worker: *Solver4* raggiunge circa 0.85, mentre *Solver2* e *Solver5* si mantengono intorno a 0.72-0.74.

In conclusione, in ottica di scalabilità, *Solver4* risulta il migliore perché raggiunge lo speedup massimo e scala bene nella configurazione con 4 worker e *partition multiplier* 4x, come mostrato in Figura 2. *Solver5* è il secondo più soddisfacente: ottiene un'efficienza molto alta con 3 worker e *partition multiplier* 3x, ma risulta meno stabile aumentando ulteriormente le partizioni.